

Day 18: MLOps for Production AI Systems

1. Five-line beginner summary

MLOps means applying DevOps-style discipline to machine learning systems. It helps move ML models from notebooks to real production safely. It manages data, code, models, experiments, deployment, monitoring, and governance. Unlike normal software, ML systems can fail because data changes over time. A good MLOps pipeline makes AI systems reliable, repeatable, auditable, and compliant.

2. Descriptive Notes

1. What MLOps is

MLOps = Machine Learning Operations

It is a set of practices used to manage the complete lifecycle of ML models.

It combines:

Area	Meaning
Machine Learning	Training, validating, improving models
DevOps	CI/CD, automation, deployment
Data Engineering	Data pipelines, validation, quality
Governance	Audit, approval, compliance, responsible AI
Monitoring	Checking model and data behavior after deployment

Simple meaning:

| MLOps helps us build, deploy, monitor, and maintain ML models in production.

In your IBM AI/GenAI preparation, MLOps connects strongly with:

Topic	MLOps connection
MLflow	Experiment tracking and model registry
Delta Lake	Reliable data pipelines and time travel
Databricks	Training, jobs, MLflow, deployment
RAG	Monitoring retrieval quality and hallucination
LangChain/LangGraph	Agent workflow monitoring
REST APIs	Real-time model serving
CI/CD	Automated deployment

2. Why ML models need lifecycle management

Normal software usually follows fixed rules.

Example:

```
if user_age >= 18:
    allow_registration
```

If the code is correct, it will usually keep working.

But ML models learn patterns from data.

Example:

```
Predict whether customer will leave company
```

The model depends on:

Dependency	Risk
Training data	Data may become old
Features	Feature meaning may change
Model logic	Model may become inaccurate
Business behavior	Customer behavior may change
External factors	Market, season, rules, policies may change

So ML models need lifecycle management because:

1. Data changes.
2. Model performance changes.
3. Business rules change.
4. Compliance requirements change.
5. Models need retraining.
6. Old model versions may need rollback.
7. Predictions must be explainable and auditable.

Simple example:

A loan approval model trained in 2023 may not work well in 2026 because customer income patterns, interest rates, regulations, and repayment behavior may have changed.

3. CI/CD for ML

In normal software:

```
Code change → Test → Build → Deploy
```

In ML:

```
Code change + Data change + Model change → Validate → Train → Evaluate →  
Register → Deploy → Monitor
```

CI/CD for ML has more moving parts.

CI for ML

CI means **Continuous Integration**.

It checks whether new code, data pipeline, or model logic is safe.

CI may include:

CI Check	Purpose
Unit tests	Check functions
Data validation	Check input data quality
Feature validation	Check feature logic
Training pipeline test	Check training does not fail
Model evaluation	Check model quality
Security scan	Check vulnerabilities
Bias check	Check unfair behavior

CD for ML

CD means **Continuous Delivery / Continuous Deployment**.

It automates model release.

CD may include:

CD Step	Purpose
Package model	Prepare model artifact
Register model	Store in model registry
Approval gate	Human approval if required
Deploy to staging	Test before production
Smoke test	Check API works
Deploy to production	Serve model
Monitor	Watch model health

4. Model testing

Model testing means checking whether the ML model behaves correctly before production.

There are different types of tests.

A. Unit tests

Test small functions.

Example:

Does preprocessing remove null values correctly?
Does feature scaling work correctly?
Does tokenizer handle empty text?

B. Data tests

Check input data.

Example:

Age should not be negative.
Salary should not be null.
Transaction amount should be greater than 0.

C. Model quality tests

Check model metrics.

Example:

Accuracy should be above 85%.
F1-score should be above 0.80.
RMSE should be below allowed threshold.

D. Bias and fairness tests

Check whether model behaves unfairly for certain groups.

Example:

Loan approval model should not unfairly reject one demographic group.

E. Robustness tests

Check how model behaves with unusual input.

Example:

Empty text
Very long text
Missing values
Outliers
Wrong format

F. API tests

Check deployed model endpoint.

Example:

API should return prediction within 300 ms.
API should return valid JSON.
API should handle bad input gracefully.

5. Data validation

Data validation checks whether incoming data is correct, complete, and usable.

ML models are highly dependent on data quality.

Bad data can produce bad predictions.

Common data validation checks

Check	Example
Schema check	Column names should match expected schema
Data type check	age should be integer
Range check	age should be between 0 and 120
Missing value check	income should not be null
Duplicate check	same transaction should not appear twice
Category check	country should be from allowed list
Freshness check	data should be updated today
Volume check	data size should not suddenly drop

Easy example

Expected training data:

<code>customer_id, age, income, city, churn</code>
--

Bad incoming data:

<code>customer_id, age, salary, city, churn</code>
--

Problem:

The column `income` is missing and replaced by `salary`.

Without validation, the training pipeline or prediction API may fail.

6. Model validation

Model validation checks whether the trained model is good enough for deployment.

It answers:

<code>Is this model better than the current model?</code>
<code>Is it safe to deploy?</code>
<code>Does it meet business and compliance requirements?</code>

Common model validation checks

Validation	Meaning
Accuracy/F1/RMSE check	Model quality

Validation	Meaning
Baseline comparison	Is new model better than old model?
Overfitting check	Train score vs test score
Bias check	Fairness across groups
Explainability check	Can we explain predictions?
Latency check	Is model fast enough?
Memory check	Can it run in production?
Stability check	Does model behave consistently?

Example

Current production model:

F1-score = 0.82
Latency = 200 ms

New model:

F1-score = 0.84
Latency = 900 ms

Even though F1-score improved, latency became too high.

Decision:

Do not directly promote to production.
Optimize model or use batch inference.

7. Model deployment

Model deployment means making the trained model available for real use.

There are several deployment patterns.

A. REST API deployment

Used for real-time prediction.

Example:

User enters customer details → API returns churn probability

Good for:

Fraud detection
Recommendation
Chatbot response
Loan eligibility

B. Batch deployment

Used when predictions are generated for many records together.

Example:

Every night predict churn risk for all customers.

Good for:

Marketing campaigns
Risk scoring
Sales forecasting
Inventory planning

C. Streaming deployment

Used when predictions happen continuously on event streams.

Example:

Every transaction is scored for fraud in near real time.

Good for:

Fraud detection
IoT monitoring
Real-time alerts

D. Edge deployment

Model runs on local device.

Example:

Mobile app image recognition
Factory camera defect detection

8. Batch inference vs real-time inference

Point	Batch Inference	Real-time Inference
Meaning	Predict many records together	Predict one/few records immediately
Timing	Scheduled	On demand
Speed need	Less strict	Very strict
Example	Daily churn prediction	Fraud detection during payment
Output	Stored in table/file	Returned through API
Infrastructure	Jobs, Spark, Databricks	REST API, Kubernetes, model server
Best for	Reports, campaigns, planning	User-facing apps

Easy example

Batch inference

```
Every night:  
Read 10 lakh customers  
Predict churn score  
Save result in database  
Marketing team uses result next morning
```

Real-time inference

```
Customer logs in  
App sends customer details to API  
Model immediately predicts next best offer  
App shows offer within 200 ms
```

9. Model monitoring

Model monitoring means watching model behavior after deployment.

A model can be good during training but become poor in production.

Monitoring checks:

Monitoring Type	What it checks
Data monitoring	Is input data changing?
Prediction monitoring	Are outputs unusual?
Performance monitoring	Is accuracy dropping?
Drift monitoring	Is data/concept changing?
System monitoring	Is API fast and healthy?
Business monitoring	Is model helping business KPIs?
Bias monitoring	Is model becoming unfair?

Example

A fraud detection model usually predicts:

```
2% transactions as suspicious
```

Suddenly it predicts:

```
25% transactions as suspicious
```

This may indicate:

```
Data issue  
Fraud pattern change  
Feature pipeline bug  
Concept drift
```

10. Data drift

Data drift happens when production input data becomes different from training data.

Example

Training data:

Most customers age: 25 to 45

Production data after 1 year:

Most customers age: 45 to 65

The model may not perform well because the input population changed.

Common causes of data drift

Cause	Example
Business change	New customer segment
Seasonal change	Festival shopping pattern
Market change	Economic slowdown
Product change	New app design changes user behavior
Data pipeline change	Feature calculation changed
External event	Pandemic, regulation, competition

Data drift detection examples

You can compare:

Training feature distribution vs production feature distribution

Example:

Average transaction amount during training = ₹1,200
Average transaction amount in production = ₹4,800

This may indicate drift.

11. Concept drift

Concept drift happens when the relationship between input and output changes.

This is more serious than data drift.

Simple meaning

Data drift:

Input data changed.

Concept drift:

Meaning/pattern changed.

Example

Old pattern:

High salary customers usually repay loans.

New pattern:

High salary customers are also defaulting because of market slowdown.

The input may look similar, but the relationship between features and target has changed.

Another example

Spam detection:

Old spam words:

free money, lottery, winner

New spam words:

crypto bonus, urgent KYC, wallet verification

The concept of spam changes over time.

12. Performance monitoring

Performance monitoring checks whether the deployed model and system are working properly.

A. Model performance metrics

For classification:

Accuracy
Precision
Recall
F1-score
ROC-AUC

For regression:

MAE
MSE
RMSE
R2 score

For GenAI/RAG:

Groundedness
Faithfulness
Relevance
Answer correctness
Hallucination rate
Retrieval precision
Retrieval recall
User feedback score

B. System performance metrics

Latency
Throughput
Error rate
CPU usage
Memory usage
API availability
Request count
Timeout count

C. Business performance metrics

Conversion rate
Fraud loss reduction
Customer retention
Manual effort reduction
Support ticket resolution time

A model should not only have good ML metrics. It should also help business outcomes.

13. Governance frameworks

Governance means defining rules, controls, approvals, and responsibilities around AI systems.

In enterprise AI, governance answers:

Who approved the model?
Which data was used?
Which model version is running?
Was bias tested?
Can predictions be explained?
Can we roll back?
Is the model compliant?

Common governance components

Governance Area	Purpose
Model registry	Track model versions
Approval workflow	Control promotion to production
Access control	Limit who can deploy

Governance Area	Purpose
Audit logs	Record all actions
Risk classification	Identify high-risk models
Explainability	Understand model decisions
Bias testing	Check fairness
Documentation	Model cards, data cards
Monitoring policy	Define alerts and retraining
Rollback policy	Return to previous safe version

Example governance flow

Data Scientist trains model MLflow logs experiment Model is registered Validation checks run Risk team reviews model Model is approved for staging Production owner approves release Model is deployed Monitoring starts Audit logs are stored

14. Auditability

Auditability means being able to answer:

What happened? When did it happen? Who did it? Why was it done? Which version was used? What data was used? What was the result?
--

In MLOps, auditability includes:

Item	Example
Data version	Training data snapshot from 2026-07-01
Code version	Git commit ID
Model version	churn_model v12
Experiment run	MLflow run ID
Parameters	learning_rate = 0.01
Metrics	F1-score = 0.86
Approver	Risk manager approved

Item	Example
Deployment time	2026-07-08 10:30 AM
Prediction logs	Input, output, timestamp
Monitoring alerts	Drift detected on income feature

Why auditability matters

Suppose a bank rejects a loan application.

The bank may need to explain:

Which model version made this decision?
 What input features were used?
 Was the model approved?
 Was it biased?
 Was the prediction logged?

Without auditability, enterprise AI becomes risky.

15. Compliance

Compliance means following laws, regulations, company policies, and industry rules.

AI systems may deal with:

Personal data
 Financial decisions
 Healthcare data
 Hiring decisions
 Customer profiling
 Sensitive information

So companies need controls.

Compliance examples

Area	Compliance Concern
Banking	Fair lending, explainability, audit trails
Healthcare	Patient privacy, safety
Insurance	Non-discrimination, documentation
HR	Fairness in hiring decisions
Customer service	Privacy and consent
GenAI	Data leakage, hallucination, unsafe output

Compliance controls in MLOps

Data access control
 PII masking

Model approval workflow
Explainability reports
Bias testing
Prediction logging
Human review for high-risk decisions
Retention policy
Security testing

16. Responsible AI basics

Responsible AI means building AI systems that are safe, fair, transparent, and human-centered.

Key principles

Principle	Meaning
Fairness	Avoid unfair bias
Explainability	Explain model decisions
Transparency	Be clear when AI is used
Privacy	Protect user data
Robustness	Work reliably under different conditions
Accountability	Humans remain responsible
Safety	Avoid harmful outputs
Governance	Follow rules and approval processes

Example

A resume screening model should not unfairly reject candidates because of gender, age, college name, or location.

Responsible AI checks may include:

Bias testing
Feature review
Human-in-the-loop approval
Explainability report
Monitoring after deployment

3. Easy Examples

Example 1: Customer churn prediction

Business goal:

Predict which customers may leave the company.

MLOps lifecycle:

Collect customer data
Validate data
Train churn model
Track experiment in MLflow
Register best model
Deploy model as batch job
Generate churn scores daily
Monitor drift and performance
Retrain when accuracy drops

Example 2: Fraud detection

Business goal:

Detect suspicious transactions.

Deployment type:

Real-time inference

Why?

Because fraud must be detected immediately during payment.

MLOps concerns:

Low latency
High recall
Real-time monitoring
Concept drift detection
Rollback strategy
Audit logs

Example 3: RAG-based internal policy assistant

Business goal:

Answer employee questions using company policy documents.

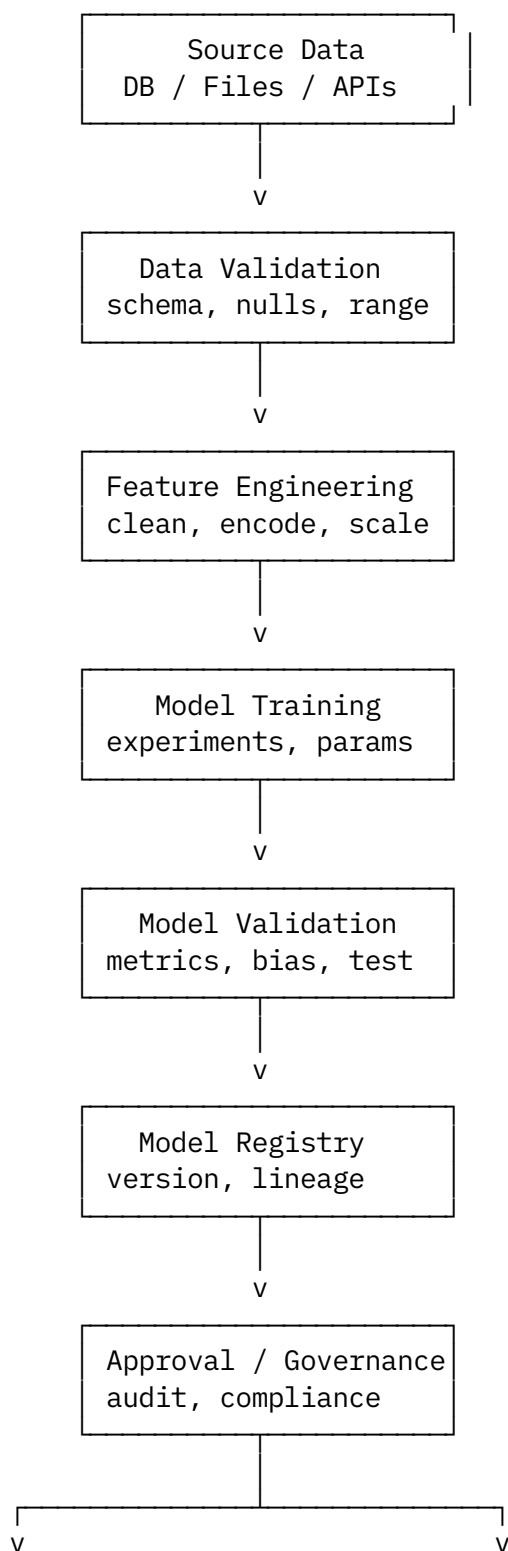
MLOps for GenAI/RAG:

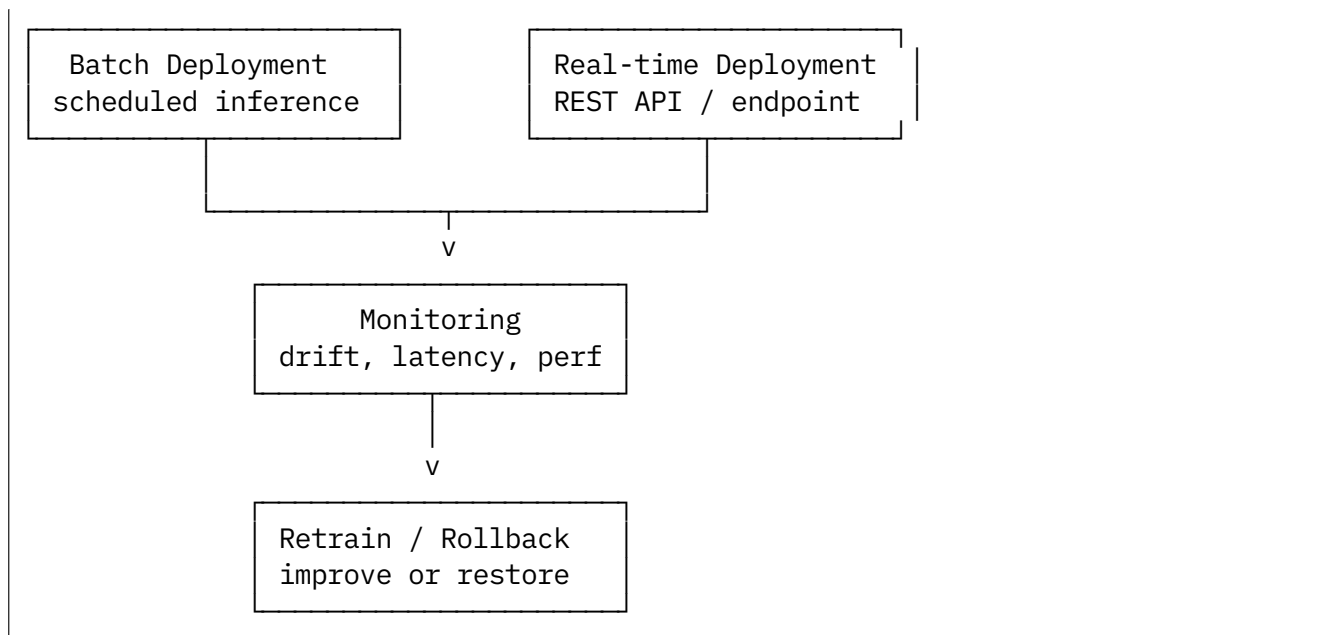
Validate documents
Chunk documents
Create embeddings
Store in vector database
Track retrieval quality
Evaluate groundedness
Monitor hallucination
Track user feedback
Update index when policies change

Important metrics:

Answer relevance
Groundedness
Citation accuracy
Hallucination rate
Retrieval precision
User satisfaction

4. ASCII Diagram Showing MLOps Pipeline





5. Pseudocode for CI/CD Model Deployment

START CI/CD PIPELINE

WHEN code is pushed to Git:

STEP 1: Run code quality checks

- check formatting
- run linting
- scan for security issues

STEP 2: Run unit tests

- test preprocessing functions
- test feature engineering functions
- test model utility functions

STEP 3: Validate training data

- check required columns
- check data types
- check missing values
- check duplicate records
- check allowed value ranges

IF data validation fails:

STOP pipeline
send alert to data team

STEP 4: Train model

- load training data
- create features
- train model
- log parameters
- log metrics
- save model artifact

STEP 5: Validate model

- compare model against baseline
- check accuracy / F1 / RMSE
- check bias metrics
- check latency
- check memory usage

IF model quality is below threshold:

STOP pipeline
mark model as rejected

STEP 6: Register model

- save model in model registry
- assign new version number
- store metrics and lineage

STEP 7: Deploy to staging

- create test endpoint
- run smoke tests
- run integration tests

IF staging tests fail:

STOP pipeline
notify ML engineer

STEP 8: Approval gate

- request approval from model owner
- request approval from governance team if high-risk

IF approval is granted:

deploy model to production

ELSE:

keep model in staging

STEP 9: Production validation

- check endpoint health
- check sample predictions
- check latency and error rate

STEP 10: Start monitoring

- monitor data drift
- monitor concept drift
- monitor prediction quality
- monitor business KPIs

END PIPELINE

6. Pseudocode for Monitoring Model Drift

START MODEL MONITORING JOB

SCHEDULE:

Run every day at 1 AM

STEP 1: Load reference data

- load training dataset statistics
- load expected feature distribution

STEP 2: Load production data

- collect yesterday's prediction inputs
- collect yesterday's prediction outputs
- collect actual labels if available

STEP 3: Check data drift

FOR each feature:

compare training distribution with production distribution

IF difference is greater than allowed threshold:

mark feature as drifted

STEP 4: Check prediction drift

- compare old prediction distribution with new prediction distribution
- check if model is predicting too many positives or negatives

STEP 5: Check model performance

IF actual labels are available:

calculate accuracy
calculate precision
calculate recall
calculate F1-score

IF performance is below threshold:

mark model as degraded

STEP 6: Check system performance

- calculate average latency
- calculate error rate
- calculate request count

IF latency is too high OR error rate is too high:

create system alert

STEP 7: Decide action

IF severe drift detected:

send alert to ML team
trigger retraining pipeline

IF model performance drops badly:

compare with previous production model
rollback if needed

IF no issue detected:

store monitoring report

STEP 8: Save audit logs

- monitoring date
- drift score
- performance metrics
- alerts generated
- action taken

END MONITORING JOB

7. MLOps Lifecycle in Simple Stages

Stage 1: Data preparation

Collect data
Clean data
Validate data
Create features
Version the dataset

Important tools/concepts:

SQL
Spark
Delta Lake
Data quality checks
Feature store

Stage 2: Model development

Train multiple models
Try different parameters
Track experiments
Compare metrics
Select best model

Important tools/concepts:

Python
Scikit-learn
PyTorch
TensorFlow
MLflow
Databricks

Stage 3: Model registration

Save best model
Create model version
Store metrics
Store artifacts
Store lineage

Important tools/concepts:

MLflow Model Registry
Model cards
Approval workflow

Stage 4: Model deployment

Deploy as API
Deploy as batch job
Deploy as streaming service
Deploy to staging first
Deploy to production after approval

Important tools/concepts:

Docker
Kubernetes
REST API
CI/CD pipeline
Databricks Jobs
Model serving

Stage 5: Model monitoring

Monitor data drift
Monitor concept drift
Monitor accuracy
Monitor latency
Monitor business KPIs
Monitor fairness

Important tools/concepts:

Logs
Dashboards
Alerts
Drift reports
Retraining triggers

Stage 6: Retraining or rollback

If model becomes weak:
 retrain with new data

If new model fails:
 rollback to previous stable model

Important tools/concepts:

```
Model versioning
Production stage
Rollback policy
Canary deployment
Blue-green deployment
```

8. Important MLOps Concepts for Interview

Model registry

A model registry stores and manages model versions.

Example:

```
churn_model
  version 1: development
  version 2: staging
  version 3: production
```

It helps answer:

```
Which model is in production?
Who approved it?
What metrics did it have?
Can we roll back?
```

Model lineage

Model lineage shows the full history of a model.

Example:

```
Training Data Version → Code Version → Experiment Run → Model Version →
Deployment
```

It helps with debugging and audit.

Rollback strategy

Rollback means returning to a previous stable model when the new model fails.

Example:

```
Production model v5 has high error rate.
Rollback to model v4.
Investigate v5 issue.
```

Rollback is important because ML models can fail silently or suddenly.

Human-in-the-loop

For high-risk AI systems, humans should review important decisions.

Example:

```
AI suggests loan rejection
Human officer reviews before final rejection
```

This is important for responsible AI and compliance.

Canary deployment

Canary deployment means releasing a new model to a small percentage of users first.

Example:

```
Model v10 gets 5% traffic
Model v9 gets 95% traffic
If v10 performs well, increase to 25%, then 50%, then 100%
```

Champion-challenger model

Current production model is the **champion**.

New candidate model is the **challenger**.

Example:

```
Champion model: v7
Challenger model: v8

Compare both on live or recent data.
Promote challenger only if better.
```

9. MLOps for GenAI Systems

For GenAI, MLOps becomes broader. It is often called **LLMOps** or **GenAIOps**.

In GenAI applications, we monitor not only model accuracy but also:

```
Prompt quality
Retrieval quality
Hallucination
Toxicity
Cost
Latency
Token usage
User feedback
Groundedness
Safety
```

Example: RAG system monitoring

For a RAG chatbot:

Component	What to monitor
Documents	Are documents updated?
Chunks	Are chunks correct size?
Embeddings	Are embeddings refreshed?
Retriever	Is it finding relevant chunks?
LLM	Is answer grounded?
Prompt	Is prompt causing bad output?
User feedback	Are users satisfied?
Cost	Are token costs increasing?

10. Simple Comparison: DevOps vs MLOps vs LLMOps

Area	DevOps	MLOps	LLMOps
Main asset	Code	Code + data + model	Prompt + model + data + retrieval
Testing	Unit/integration tests	Data/model tests	Prompt, safety, hallucination tests
Deployment	Application	ML model	LLM app/RAG/agent
Monitoring	Logs, latency, errors	Drift, accuracy, latency	Hallucination, cost, relevance
Versioning	Code version	Code + data + model version	Prompt + model + index + data version
Failure reason	Code bug	Data/model drift	Bad retrieval, hallucination, unsafe output

11. Common Mistakes

1. Treating ML deployment like normal software deployment

ML models depend on data. Even if code does not change, model performance can drop.

2. Not validating data

Bad input data can silently damage training and prediction.

Example:

income column suddenly has null values
--

Without validation, model output may become unreliable.

3. Not tracking experiments

If experiments are not tracked, you cannot answer:

Which model was best?
Which parameters were used?
Which data was used?
Can we reproduce the model?

4. Deploying model without baseline comparison

A new model should be compared against the current production model.

Do not deploy only because the training score looks good.

5. Ignoring drift

A model can become outdated even if it was excellent during training.

Monitor:

Data drift
Concept drift
Prediction drift
Performance drift

6. No rollback plan

Always keep previous stable model versions.

Production AI should have:

Rollback strategy
Version history
Deployment logs
Approval records

7. Monitoring only technical metrics

Latency and uptime are not enough.

Also monitor:

Accuracy
Business impact
Fairness
Drift
User feedback

8. No governance for high-risk models

Models used in banking, healthcare, insurance, HR, or legal workflows need stronger review.

9. Not logging predictions

Without prediction logs, debugging becomes difficult.

But logs should be handled carefully to protect privacy.

10. Ignoring Responsible AI

A model can be accurate but unfair, unsafe, or non-compliant.

Responsible AI is not optional in enterprise AI.

Final takeaway

MLOps is the production discipline for AI systems. It ensures that models are not just trained successfully, but also deployed safely, monitored continuously, governed properly, and improved over time. For IBM AI/GenAI roles, you should understand MLOps as the bridge between **data science experiments** and **real enterprise AI platforms**.